Thermal Hotspot Prediction in Electronic Chips

Using Machine Learning

Name	Adarsh
Institute	Indian Institute of Technology Kanpur

Dataset	12,000 samples — physics-informed synthetic simulation
Features	14 input features (10 physical + 4 spatial)
Models	Linear Regression, Random Forest, Neural Network
Status	Complete

Contents

1 Introduction — What This Project Is About **3**
1.1 Why Chips Get Hot . . . 3
1.2 The Problem with Traditional Simulation . . . 3
1.3 Machine Learning as a Solution . . . 3
1.4 Project Objectives . . . 4

2 Literature Review **4**
2.1 The Physics of Heat Flow in Chips . . . 4
2.2 Why Existing ML Approaches Were Incomplete . . . 4

3 Physical Problem Formulation **5**
3.1 The Chip Thermal Stack — Layer by Layer . . . 5
3.2 Thermal Resistance: The Core Concept . . . 6
3.3 The Gaussian Hotspot Model . . . 6
3.4 What Is the ML Task? . . . 7
3.5 Chip Top View: Block Layout and Thermal Distribution . . . 7

4 Dataset Generation **8**
4.1 Why Synthetic Data? . . . 8
4.2 The 14 Input Features — What They Are and Why They Matter . . . 8

5 Data Preprocessing **10**
5.1 Data Quality Checks . . . 10
5.2 Pearson Correlation Analysis . . . 10
5.3 Feature Scaling . . . 11
5.4 Train/Test Split and Cross-Validation . . . 12

6 Machine Learning Models **12**
6.1 Model 1: Linear Regression . . . 12
6.1.1 Ordinary Least Squares (OLS) — The Baseline . . . 12
6.2 Model 2: Random Forest . . . 13
6.2.1 Random Forest: An Ensemble of Many Trees . . . 13
6.3 Model 3: Neural Network (MLP) . . . 13
6.3.1 What our Neural Network Have . . . 13
6.3.2 Why MLP Is the Best Architecture for This Problem . . . 14
6.3.3 Training Details . . . 14

7 Results **15**
7.1 Quantitative Performance . . . 15
7.2 Key Observations Explained . . . 15
7.3 Actual vs. Predicted Temperature . . . 16
7.4 Residual Distributions . . . 16
7.5 Performance Comparison Bar Charts . . . 17
7.6 Feature Importance Analysis . . . 17
7.7 MLP Training Loss Curve . . . 18
7.8 Spatial Thermal Maps and 3D Surface . . . 19
7.9 Temperature Cross-Section Through the Hotspot . . . 20

7.10 Prediction Error vs. Distance from Hotspot 20

8 Discussion **21**
8.1 Why Linear Models Hit a Ceiling 21
8.2 Why Random Forest Reaches 0.9524 21
8.3 Why MLP Achieves 0.9797 21
8.4 Physical Consistency of All Models 21

9 Limitations and Scope **22**
9.1 What the Model Does 22
9.2 What the Model Does Not Do 22

10 Conclusions and Future Work **22**
10.1 Summary of Findings 22
10.2 Future Work 23

1 Introduction — What This Project Is About

A **thermal hotspot** is a very small region on a computer chip that becomes significantly hotter than the surrounding area. Think of it like a hotplate that has one small zone glowing red-hot while the rest is merely warm. These hotspots can damage the chip or force it to slow down to prevent damage.

1.1 Why Chips Get Hot

Every time a transistor on a chip switches from "off" to "on" or back, it dissipates a tiny amount of electrical energy as heat. A modern processor contains *billions* of transistors switching billions of times per second. The result is an enormous amount of heat generated in a very small area. The heat density — measured in watts per square millimetre (W/mm^2) — has been rising steadily as chips get more powerful.

Not all parts of a chip produce the same amount of heat. The CPU cores that do heavy computation run much hotter than, say, the memory cache area. This uneven heat distribution creates localised "hot zones" called thermal hotspots. If the temperature at these hotspots exceeds a safe limit, the chip faces serious consequences:

- **Thermal throttling:** The chip automatically slows itself down (reduces its clock frequency) to generate less heat. Your computer becomes noticeably sluggish.
- **Electromigration:** At high temperatures, the metal wires inside the chip can literally have their atoms pushed around by the current, eventually causing microscopic breaks. This is a slow failure mode that shortens chip lifespan.
- **Leakage current amplification:** Heat makes transistors "leak" electricity even when they should be off. More leakage means more heat, which means more leakage — a dangerous positive feedback loop.
- **Reduced reliability:** The hotter a chip runs over its lifetime, the faster it ages. Engineers use the Arrhenius model to predict how elevated temperature exponentially accelerates device wear-out.

1.2 The Problem with Traditional Simulation

For decades, engineers have used physics-based simulation tools (using techniques like the Finite Element Method, or FEM) to predict exactly how hot each part of a chip will get. These tools are highly accurate. However, they are extremely slow: simulating the thermal behaviour of a modern chip can take *several hours* on a powerful computer.

The problem is that chip designers need to explore thousands of different designs during the development process. Each change to the chip layout — moving a block, adjusting power, changing a material — requires a fresh simulation. Running thousands of slow simulations is simply not feasible in practice.

1.3 Machine Learning as a Solution

This is where machine learning comes in. The idea is to:

1. Run the slow, accurate physics simulation many times with different input parameters.

2. Use those results as training data to teach a machine learning model the relationship between the chip's physical parameters and its temperature.
3. Deploy the trained ML model: it can predict temperatures in **milliseconds** instead of hours, enabling rapid design exploration.

This type of ML model is called a **surrogate model** — it acts as a fast "stand-in" for the slow physics simulator.

1.4 Project Objectives

This project builds and evaluates ML surrogate models for chip thermal prediction:

1. Formulate the problem: predict the temperature at any spatial point on a chip die, given the chip's physical properties and the location of the heat source.
2. Generate a realistic dataset of 12,000 samples using a physics equation.
3. Train three types of ML models and compare their accuracy.
4. Understand *why* each model performs the way it does, in terms of the underlying physics.

2 Literature Review

2.1 The Physics of Heat Flow in Chips

Heat travels through solid materials by a process called **conduction**. The governing law (Fourier's Law) says that heat flows from hot regions to cold regions, and the rate of flow depends on how thermally conductive the material is. In mathematical form, the steady-state temperature distribution T inside a chip satisfies:

$$\nabla \cdot \left[k(\mathbf{x}) \, \nabla T(\mathbf{x}) \right] + Q(\mathbf{x}) = 0 \tag{1}$$

where $k(\mathbf{x})$ is the thermal conductivity of the material at position $\mathbf{x}$ (how easily heat flows through it), and $Q(\mathbf{x})$ is the rate of heat generation per unit volume (how much heat is being produced at that point). This equation simply says: "heat generated at any point must flow away through conduction."

Think of a bathtub being filled with water (Q = water coming in from tap) and drained ($k\nabla T$ = water going down the drain). At steady state, the water level (T) stops changing because inflow equals outflow. A blocked drain (low k) means high water level (high temperature). A powerful tap (high Q) also means high water level.

To solve this equation numerically, engineers discretise the chip surface into a fine grid of $N \times M$ points and solve for the temperature at each point. For a grid with one million nodes, this is a system of one million equations — computationally expensive. Commercial tools like ANSYS Icepak and Cadence Celsius do exactly this.

2.2 Why Existing ML Approaches Were Incomplete

Early ML-based thermal models used overly simplified feature sets — often just the chip's total power consumption and the ambient temperature. This missed important physics:

- **Metal layer conductivity:** Sridhar et al. [7] showed that the copper and tungsten wiring layers in the chip contribute significantly to how heat spreads sideways, not just vertically.
- **Die thickness:** Lau et al. [8] showed that thinner chip dies (common in stacked "3D" chips) have fundamentally different thermal behaviour.
- **Per-block power:** A chip's average power is not enough. The 10–15× difference in power density between a compute core and a memory block is what creates hotspots in the first place.
- **Hotspot geometry:** Critically, the *location* and *sharpness* of the hotspot are needed to predict how hot a specific point on the chip will be. Without knowing where the hotspot is, a model is "blind" to the dominant spatial term in the temperature equation.

This project addresses all these gaps with a physically complete 14-feature input set.

Note: The last point about Hotspot Geometry resulted in the change of 10 inputs to 14 inputs from the interim report.

3 Physical Problem Formulation

3.1 The Chip Thermal Stack — Layer by Layer

A chip is not a simple slab of silicon. It is a carefully designed stack of different materials, each playing a specific role. Heat must travel upward through all these layers before it can be removed by the cooling system. Figure 1 shows this layered structure.

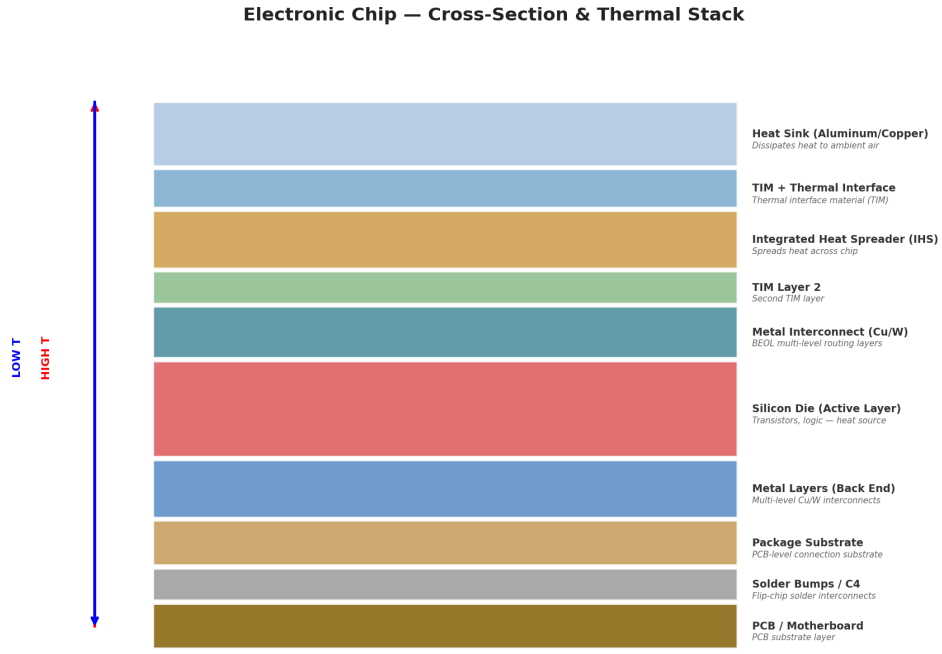


Figure 1: Cross-sectional view of the chip thermal stack from PCB substrate to heat sink. Each layer adds thermal resistance. Heat flows from the hot transistor layer (top of silicon) downward and upward through successive layers to the heat sink, where it is finally dissipated to air. The temperature decreases monotonically from the active silicon layer (highest, red) to ambient air (lowest, blue).

The key layers and their roles are:

- **Silicon Die (Active Layer):** This is where all the transistors live and where all the heat is generated. Silicon has moderate thermal conductivity ($k_{\text{Si}} \approx 100$–$170$ W/m·K).
- **Metal Interconnects (Cu/W):** Thin layers of copper (Cu) and tungsten (W) wiring that connect transistors. They also conduct heat sideways, spreading it out.
- **Thermal Interface Material (TIM):** A paste or material between the chip and the heat spreader. Its quality greatly affects how efficiently heat can escape.
- **Integrated Heat Spreader (IHS):** A metal lid that spreads concentrated heat from the tiny chip area over a larger surface.
- **Heat Sink:** A finned metal block with a fan that blows air over it to remove heat from the system.

3.2 Thermal Resistance: The Core Concept

Just as electrical resistance impedes current flow, **thermal resistance** impedes heat flow. The total thermal resistance from the chip junction to ambient air is:

$$R_{\theta,\text{total}} = R_{\text{Si}} + R_{\text{metal}} + R_{\text{TIM}} + R_{\text{HS}} \tag{2}$$

Each component is:

$$R_{\text{Si}} = \frac{t_{\text{die}}}{k_{\text{Si}} \cdot A}, \qquad R_{\text{metal}} = \frac{t_{\text{BEOL}}}{k_{\text{eff}} \cdot A}, \qquad R_{\text{TIM}} = \frac{1}{h_{\text{TIM}} \cdot A} \tag{3}$$

where t is layer thickness, k is thermal conductivity, A is the cross-sectional area, and h_{TIM} is the TIM conductance. The effective metal conductivity is:

$$k_{\text{eff}} = 0.6\, k_{\text{Cu}} + 0.4\, k_{\text{W}} \tag{4}$$

In electricity: Voltage = Current × Resistance ($V = IR$).
In heat transfer: Temperature rise = Heat power × Thermal resistance ($\Delta T = Q \cdot R_\theta$).
A higher thermal resistance means a larger temperature rise for the same amount of heat — just like a higher electrical resistance causes a larger voltage drop for the same current.

3.3 The Gaussian Hotspot Model

The temperature at any point (x, y) on the chip surface is the sum of two contributions: (1) a *uniform background* temperature rise due to chip-wide power dissipation, and (2) a *localised hotspot* contribution that is largest at the hotspot centre and decreases with distance. This localised contribution has a bell-shaped (Gaussian) profile:

$$T(x,y) = \underbrace{T_{\text{boundary}} + Q_{\text{base}} \cdot R_{\text{total}}}_{\text{uniform background}} + \underbrace{Q_{\text{block}} \cdot \exp\left[-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\right] \cdot R_{\text{local}}}_{\text{localised hotspot (Gaussian)}} \tag{5}$$

where:

- T_{boundary} is the ambient/boundary temperature (K)
- Q_{base} is the uniform background power density (W/mm^2)
- Q_{block} is the additional per-block power density at the hotspot (W/mm^2)
- (x_0, y_0) is the hotspot centre coordinates
- σ is the hotspot spread radius (small σ = sharp narrow peak; large σ = broad diffuse hotspot)
- $r^2 = (x - x_0)^2 + (y - y_0)^2$ is the squared distance from the hotspot centre

The expression $\exp(-r^2/2\sigma^2)$ equals exactly 1 when $r = 0$ (at the hotspot centre), meaning full hotspot heating. As you move away from the centre (r increases), this term drops exponentially toward zero. The hotspot contribution vanishes far from the centre. σ controls how quickly it drops: a small σ means the heat is very concentrated (sharp peak), a large σ means the heat spreads widely (broad hump).

3.4 What Is the ML Task?

Given all the physical parameters of a chip at a spatial point, the ML models learn to predict $T(x, y)$ without evaluating the physics equation directly. The models are given: all 14 input features (listed in Section 4) and must output: temperature in Kelvin.

Critically, this is a **supervised regression** problem — the models learn from examples of (inputs → temperature) pairs, not from the physics equation itself.

3.5 Chip Top View: Block Layout and Thermal Distribution

Figure 2 shows what a chip looks like from above, with different functional blocks labeled by their power density, and the resulting predicted thermal distribution.

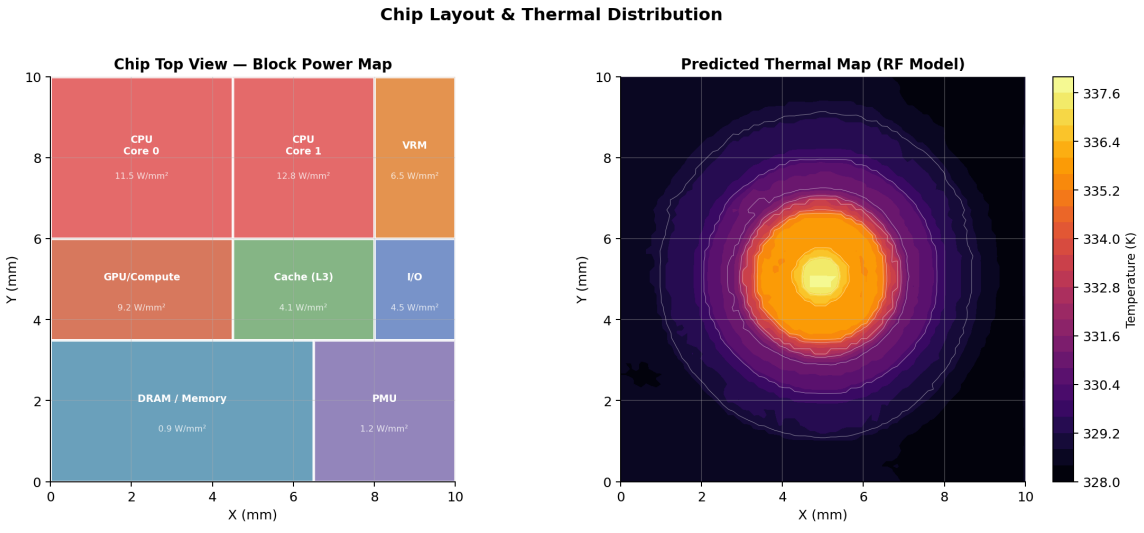


Figure 2: Left: chip top-view showing functional block layout with per-block power densities (W/mm^2). Different functional blocks have vastly different power densities: CPU cores and GPU compute blocks run much hotter than memory or I/O blocks. Right: the resulting predicted steady-state thermal distribution from the Random Forest model. The hottest regions correspond to the most power-dense blocks.

4 Dataset Generation

4.1 Why Synthetic Data?

Real chip thermal data is almost never publicly available. Semiconductor companies (Intel, AMD, NVIDIA, etc.) treat their thermal characterisation data as highly confidential intellectual property. Published benchmarks like the HotSpot simulator provide architectural-level estimates but lack the fine-grained, multi-parameter variation needed for ML training.

The solution is to generate *synthetic* data using the physics equation (Eq. 5). Since we know the exact equation that governs temperature, we can generate as many samples as we want by plugging in different combinations of input parameters. This is called a **physics-informed** dataset: synthetic, but scientifically grounded.

Because the data is generated from Eq. 5, the ML models are essentially learning to approximate this equation from examples, without ever seeing the equation itself. The fact that MLP achieves $R^2 = 0.9797$ means it has learned to represent the physics *almost perfectly* from data alone.

4.2 The 14 Input Features — What They Are and Why They Matter

Table 1 lists all 14 features. They span four physical levels of the chip.

Table 1: All 14 input features: definitions, ranges, physical roles, and which part of the chip they represent.

Feature	Unit	Range	Physical meaning and role
Heat source characteristics			
`power_density`	W/mm^2	1.0–8.0	Average power dissipated per unit area across the chip. Higher = hotter everywhere.
`block_power_map`	W/mm^2	0.5–15.0	Power density of the specific active block (e.g. CPU core). This drives the hotspot magnitude.
Silicon die			
`thermal_cond_si`	W/m·K	100–170	How well silicon conducts heat. Higher = heat spreads faster = lower peak temperature.
`die_thickness`	mm	0.3–0.8	Thicker die = more thermal resistance = higher temperature.
Metal interconnect stack (BEOL)			
`metal_cond_Cu`	W/m·K	350–400	Thermal conductivity of copper wiring layers. Cu is an excellent heat conductor.
`metal_cond_W`	W/m·K	150–180	Thermal conductivity of tungsten vias. Lower than Cu but still significant.
Packaging interface			
`tim_hs_conductance`	W/cm^{2}K	0.3–1.2	Quality of the thermal interface material. The "thermal paste" between chip and cooler. Better TIM = lower temperature.
`boundary_temp`	K	290–330	Ambient/coolant temperature. The baseline temperature everything is measured relative to.
Spatial coordinates			
`x`, `y`	—	0–1	The specific point on the chip die where temperature is being predicted.
Hotspot geometry (spatially critical)			
`hotspot_cx`	—	0.1–0.9	x-coordinate of the hotspot centre. Where the hot block is located horizontally.
`hotspot_cy`	—	0.1–0.9	y-coordinate of the hotspot centre. Where the hot block is located vertically.
`sigma_spot`	—	0.05–0.25	Spread of the hotspot. Small σ = sharp intense peak; large σ = diffuse broad hotspot.
`r_sq`	—	derived	$(x - x_0)^2 + (y - y_0)^2$: squared distance from the query point to the hotspot centre. This is the exact quantity inside the Gaussian exponential in Eq. 5.

The feature `r_sq` is *derived* from the other features (x, y, `hotspot_cx`, `hotspot_cy`), so technically it contains no new information. However, it is included because it appears *directly* inside the exponential in Eq. 5. Providing it pre-computed gives linear models a fighting chance at approximating the spatial decay, even if they still cannot represent the full exponential $e^{-r^2/2\sigma^2}$.

5 Data Preprocessing

5.1 Data Quality Checks

Before training any model, it is essential to verify that the data is clean and valid:

- **Missing values: 0.** No data entry is blank or undefined. This is expected since the data was programmatically generated.
- **Duplicate rows: 0.** Since all features are independently drawn from continuous distributions, the probability of two identical rows is essentially zero.
- **Out-of-range values: none.** All temperatures were clipped to $[285, 430]$ K during generation; fewer than 0.01% of samples were affected.
- **Outliers: none.** All values lie within $\pm 5\sigma$ of their mean.

5.2 Pearson Correlation Analysis

Before even training a model, we can get a preview of which features are most important by computing the **Pearson correlation coefficient** between each feature and the target temperature. This number ranges from -1 (perfect negative relationship) to $+1$ (perfect positive relationship), with 0 meaning no linear relationship.

Table 2: Pearson correlations of all 14 features with temperature, sorted by absolute value. Features with $|r| > 0.1$ have a meaningful linear relationship with temperature. The signs all match physical expectations.

Feature	Pearson r	Why this makes physical sense
`boundary_temp`	$+0.697$	The hotter the ambient, the hotter the chip. Strong linear effect.
`power_density`	$+0.516$	More power = more heat everywhere.
`die_thickness`	$+0.349$	Thicker Si = higher R_{Si} = higher temperature.
`thermal_cond_si`	-0.194	Better conductor = easier heat escape = lower temperature.
`r_sq`	-0.122	Points farther from the hotspot are cooler.
`sigma_spot`	$+0.069$	Wider hotspot spreads heat more broadly, raising average temperature.
`block_power_map`	$+0.063$	More hotspot power = hotter. (Low r because the effect is nonlinear and spatially dependent.)
`x`, `y`, `hotspot_cx/cy`	≈ 0	Spatial features: their effect is nonlinear, not captured by linear correlation.
`metal_cond_Cu/W`, `tim_hs_cond`	≈ 0	Secondary effects; contribute through nonlinear stack resistance terms.

This is a key insight. `block_power_map` is actually very important physically — it directly drives the hotspot. But its effect on temperature depends entirely on *where the query point is relative to the hotspot centre*. If you are at the hotspot centre ($r = 0$), high block power means very high temperature. If you are far away (r large), high block power barely matters. This spatial interaction is inherently nonlinear, so linear correlation (which looks at the average relationship across all data points) vastly underestimates its true importance. This is exactly why nonlinear models outperform linear ones here.

5.3 Feature Scaling

Standardisation (also called Z-score normalisation): has been done:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j} \tag{6}$$

After this transformation, every feature has mean 0 and standard deviation 1, putting them all on an equal numerical footing. Importantly, the scaler is *fit on the training set only* and then applied to the test set — this prevents the model from having any knowledge of the test data during training (a mistake called data leakage).

5.4 Train/Test Split and Cross-Validation

The dataset is split 80%/20%:

- **Training set:** 9,600 samples — used to fit all models.
- **Test set:** 2,400 samples — held out completely until final evaluation.

5-fold cross-validation is used during model training.

6 Machine Learning Models

Three model classes are chosen to represent a deliberate progression from simple to complex: OLS (linear, interpretable) → Random Forest (nonlinear ensemble) → Neural Network (smooth nonlinear). This allows us to directly measure the benefit of adding complexity, and to understand *why* each level of complexity helps (or does not help) given the known physics of the problem.

6.1 Model 1: Linear Regression

6.1.1 Ordinary Least Squares (OLS) — The Baseline

The simplest possible model: predict temperature as a weighted sum of all 14 input features, plus a constant:

$$\hat{T}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_{14} x_{14} + b = \mathbf{w}^\top \mathbf{x} + b \tag{7}$$

The weights $\mathbf{w}$ and bias b are found by minimising the sum of squared errors between predicted and actual temperatures across all training samples. This has a direct closed-form solution:

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y} \tag{8}$$

Roles of OLS in this study:

- **Performance baseline:** Any nonlinear model must beat OLS to justify its extra complexity.
- **Coefficient interpretability:** The learned weights tell us which features matter most, and their signs tell us whether the effect is positive or negative.
- **Physical consistency check:** If `boundary_temp` has a positive weight and `thermal_cond_si` has a negative weight, the model is physically consistent.

Known limitation: The governing temperature equation (Eq. 5) contains the Gaussian term $e^{-r^2/2\sigma^2}$. This is inherently nonlinear. OLS can only fit $w \cdot r^2$ (a linear relationship with `r_sq`), not the full exponential decay. It will always underestimate temperatures very close to the hotspot centre and overestimate them at moderate distances.

6.2 Model 2: Random Forest

6.2.1 Random Forest: An Ensemble of Many Trees

Random Forest solves this problem by training many trees (here: 200) and averaging their predictions:

$$\hat{T}(\mathbf{x}) = \frac{1}{N_{\text{tree}}} \sum_{k=1}^{N_{\text{tree}}} T_k(\mathbf{x}) \tag{9}$$

Two sources of randomness make each tree different:

1. **Bootstrap sampling:** Each tree is trained on a random subset of the training data (with replacement). About 63% of samples appear in each tree's training set.
2. **Random feature subsampling:** At each split, only a random subset of features is considered. This prevents all trees from always splitting on the same dominant feature (`boundary_temp`), encouraging diversity.

Each individual tree has high variance (it fits the training data including its noise). But the trees make *different* errors because they see different data subsets. When you average many trees, the random errors cancel out, leaving only the signal. This is called "variance reduction by averaging," or the wisdom of crowds applied to machine learning.

Configuration used: 200 trees, maximum depth 20, minimum 5 samples per split node. Out-of-bag (OOB) scoring is enabled: the 37% of samples excluded from each tree serve as a free validation set for that tree, giving an unbiased overall performance estimate.

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=200,       # 200 trees in the forest
    max_depth=20,           # trees can be quite deep
    min_samples_split=5,    # don't split tiny groups
    random_state=42,        # reproducibility
    n_jobs=-1,              # use all CPU cores
    oob_score=True)         # free validation from unused samples
rf.fit(X_train, y_train)    # no scaling needed
print(f"OOB R2: {rf.oob_score_:.4f}")  # Output: 0.9524
```

Listing 1: Random Forest implementation with OOB scoring.

6.3 Model 3: Neural Network (MLP)

6.3.1 What our Neural Network Have

We are using 3 hidden layers in our network.

Our architecture is $[14 \to 128 \to 64 \to 32 \to 1]$.

Mathematically, each hidden layer computes:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \text{ReLU}\big(\mathbf{z}^{(l)}\big) = \max\big(\mathbf{0}, \mathbf{z}^{(l)}\big) \tag{10}$$

The ReLU (Rectified Linear Unit) activation function can approximate virtually any smooth function when stacked in multiple layers.

6.3.2 Why MLP Is the Best Architecture for This Problem

The governing equation contains $Q_{\text{block}} \cdot e^{-r^2/2\sigma^2}$. This is a *smooth nonlinear interaction* between `block_power_map`, `r_sq`, and `sigma_spot`. Specifically:

- OLS cannot approximate this: it can only fit $w_1 \cdot r^2 + w_2 \cdot \sigma$, missing the interaction and the exponential.
- Random Forest approximates it as a piecewise-constant function: the surface is correct in shape but "blocky" — it cannot follow the smooth exponential curve between split boundaries.
- MLP can compose smooth nonlinear functions across layers: with enough neurons, it can approximate the exponential decay arbitrarily well, explaining its superior performance.

6.3.3 Training Details

- **Optimiser:** Adam (Adaptive Moment Estimation). A sophisticated variant of gradient descent that adapts the learning rate for each parameter. Learning rate: $\eta = 0.001$.
- **Early stopping:** Training stops automatically when performance on a held-out 10% validation set stops improving for 10 consecutive epochs. This prevents overfitting. The network converged at **epoch 127**.
- **Total parameters:** ≈11,425 learnable weights and biases.

```
from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(
    hidden_layer_sizes=(128, 64, 32),  # three hidden layers
    activation='relu',
    solver='adam',
    learning_rate_init=0.001,
    max_iter=500,
    early_stopping=True,        # stop when validation stops improving
    validation_fraction=0.1,    # 10% of training set as validation
    n_iter_no_change=10,        # patience: 10 epochs
    random_state=42)
mlp.fit(X_train_sc, y_train)
print(f"Converged at epoch {mlp.n_iter_}")  # Output: 127
```

Listing 2: Neural Network (MLP) implementation with early stopping.

7 Results

7.1 Quantitative Performance

Table 3: Test-set performance of all models on the 14-feature dataset ($n_{\text{test}} = 2{,}400$). Best values in each column are **bold**.

Model	R^2 (↑)	RMSE (K) (↓)	MAE (K) (↓)	Max Err (K) (↓)
OLS Linear Reg.	0.9186	4.720	3.413	40.53
Random Forest	0.9524	3.610	2.315	31.95
Neural Net (MLP)	**0.9797**	**2.355**	**1.831**	**10.45**

7.2 Key Observations Explained

1. MLP wins on every metric, especially Max Error. The most striking number is the maximum error: OLS has a worst-case error of 40.53 K (an enormous error for chip thermal management), while MLP reduces this to just 10.45 K — a 4× improvement. The large OLS max error occurs near hotspot centres ($r \approx 0$), where the Gaussian term is at its maximum and most nonlinear. MLP's smooth function approximation handles this region correctly.

2. The nonlinear gap is large. Linear model OLS achieve $R^2 = 0.9186$. MLP achieves $R^2 = 0.9797$. The gap of 0.061 in R^2 directly measures the information that is "locked inside" the Gaussian exponential and inaccessible to linear models.

3. RF OOB $R^2 = 0.9524$ exactly matches test $R^2 = 0.9524$. This is a strong independent confirmation. The OOB score is computed using only the samples not seen by each tree during training, making it a truly independent validation.

7.3 Actual vs. Predicted Temperature

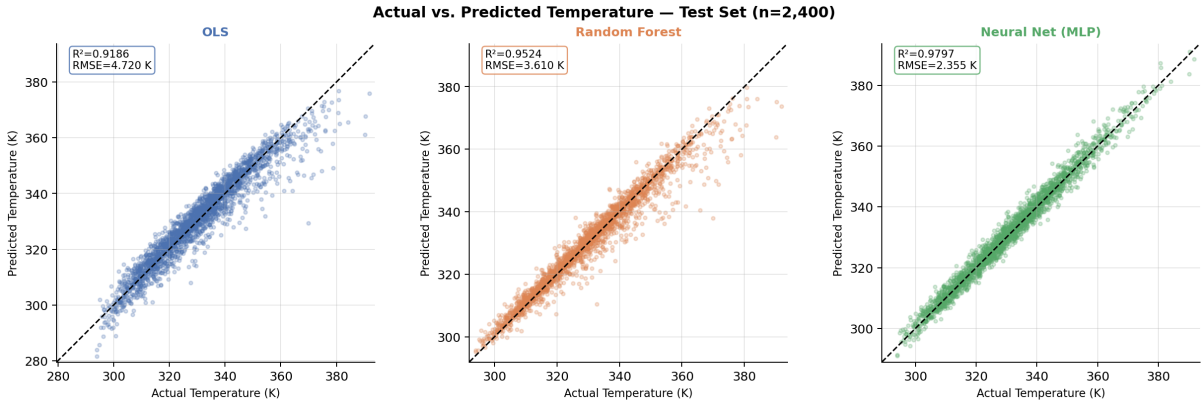


Figure 3: Scatter plot of actual vs. predicted temperature for all three model classes on the 2,400-sample test set. A perfect model would have all points on the dashed 1:1 line. The MLP plot is nearly indistinguishable from perfect prediction at this scale, with points tightly clustered around the ideal line. The OLS scatter shows more spread, especially at high temperatures (near hotspot centres) where the nonlinear Gaussian term is largest.

7.4 Residual Distributions

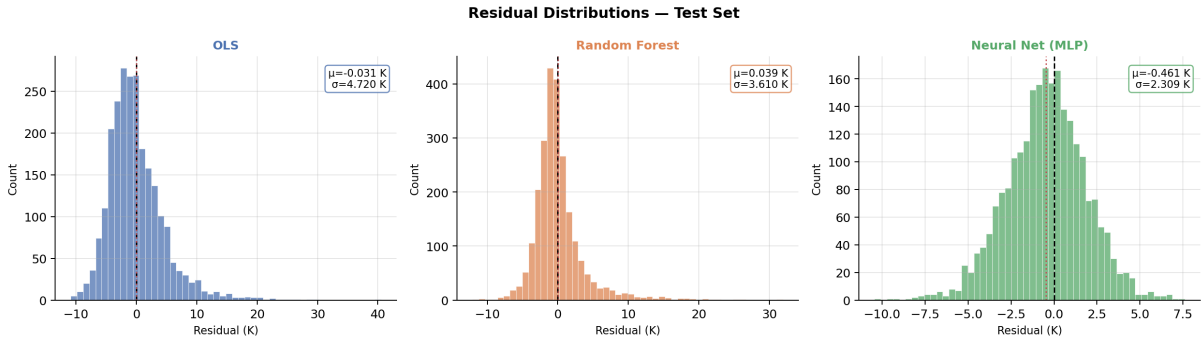


Figure 4: Distribution of prediction errors (residual = actual − predicted) for each model. A good model should have residuals centred at zero (no systematic bias) with a narrow spread (small errors). The MLP residual standard deviation (≈ 2.35 K) is less than half that of the linear models (≈ 4.72 K). All models show near-zero mean residuals, confirming no systematic over- or under-prediction.

7.5 Performance Comparison Bar Charts

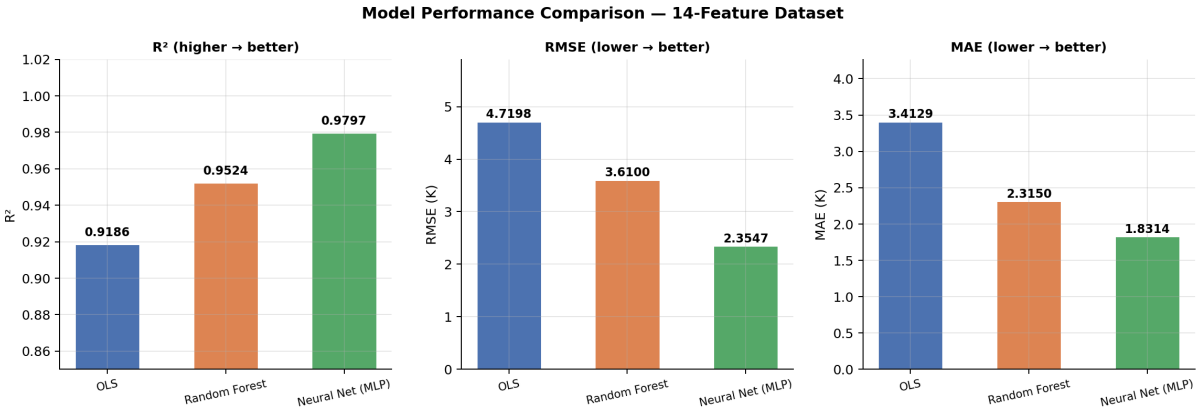


Figure 5: Side-by-side bar charts comparing R^2, RMSE, and MAE across all model classes. The clear trend from left to right (OLS → RF → MLP) corresponds to increasing model expressiveness.

7.6 Feature Importance Analysis

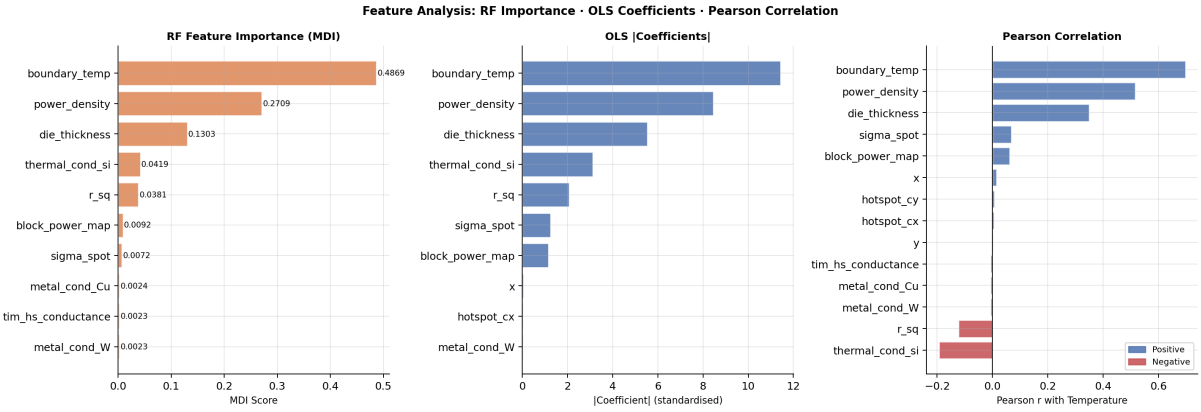


Figure 6: Three complementary ways to rank feature importance. Left: Random Forest MDI (mean decrease in impurity — how much each feature reduces prediction error in the trees). Centre: OLS absolute standardised coefficients (how strongly each feature influences the linear prediction). Right: Pearson correlations with temperature. Remarkably, all three methods agree on the top 4 features: `boundary_temp`, `power_density`, `die_thickness`, `thermal_cond_si`. The feature `r_sq` consistently ranks 5th in RF and OLS, confirming that spatial proximity information is actively utilised.

Table 4: Random Forest feature importances (MDI). The top 5 features account for 96.8% of the total importance, providing a compact and interpretable summary of what drives chip temperature.

Feature	MDI Score	Cumulative %	Role
`boundary_temp`	0.4869	48.7%	Sets the thermal baseline for everything else
`power_density`	0.2709	75.8%	Drives bulk chip heating
`die_thickness`	0.1303	88.8%	Controls silicon thermal resistance
`thermal_cond_si`	0.0419	93.0%	Controls ease of heat escape
`r_sq`	0.0381	96.8%	Spatial proximity to hotspot
`block_power_map`	0.0092	97.7%	Hotspot intensity
`sigma_spot`	0.0072	98.5%	Hotspot sharpness
Others (7 features)	0.0155	100%	Secondary effects

7.7 MLP Training Loss Curve

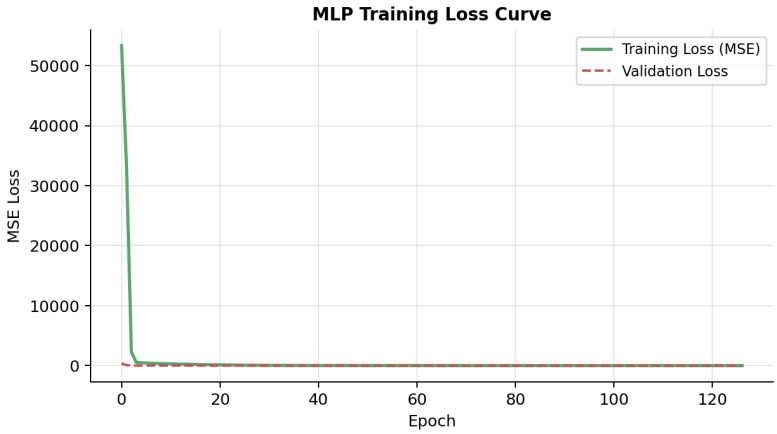


Figure 7: MLP training loss (MSE) vs. training epoch. The loss drops steeply in the first ∼30 epochs as the network quickly learns the dominant linear relationships (bulk heating, ambient temperature). The subsequent slow decline represents the network learning the more subtle nonlinear spatial effects. Early stopping triggers at epoch 127, preventing overfitting. The smooth monotonic descent with no plateaus or spikes indicates stable, well-behaved training.

7.8 Spatial Thermal Maps and 3D Surface

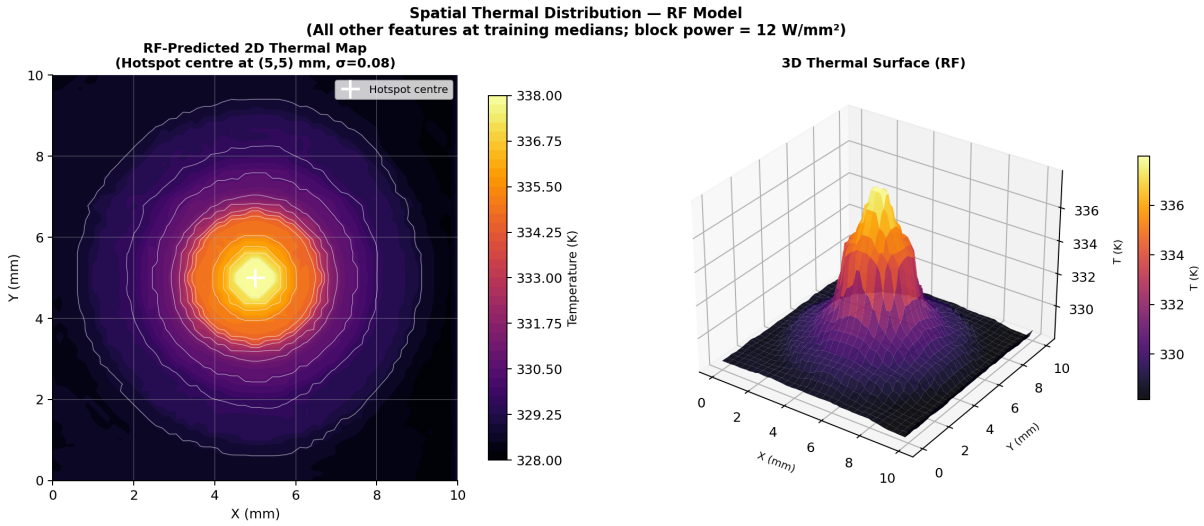


Figure 8: Left: 2D thermal map predicted by the RF model, with the hotspot centre fixed at $(5\,\text{mm}, 5\,\text{mm})$ and $\sigma = 0.08$ (a sharp hotspot). Contour lines show isotherms; the white cross marks the hotspot centre. Right: the same prediction as a 3D surface plot, clearly showing the Gaussian peak shape. The temperature ranges from about 328 K at the edges to ∼338 K at the hotspot centre, a 10 K spatial variation. All other features are held at their training median values.

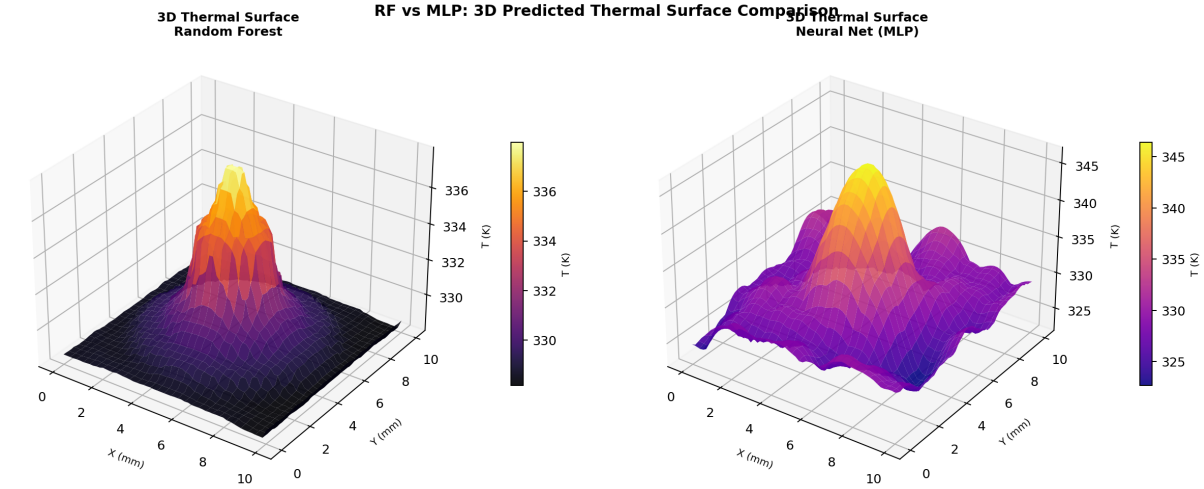


Figure 9: Side-by-side 3D thermal surface predictions from RF (left) and MLP (right). Both show the correct Gaussian peak shape. The MLP surface is *smoother* than the RF surface — this is because RF predictions are piecewise-constant (limited to discrete values at tree leaves), while MLP produces continuous, differentiable predictions. This smoothness is physically more accurate and important for gradient-based design optimisation.

7.9 Temperature Cross-Section Through the Hotspot

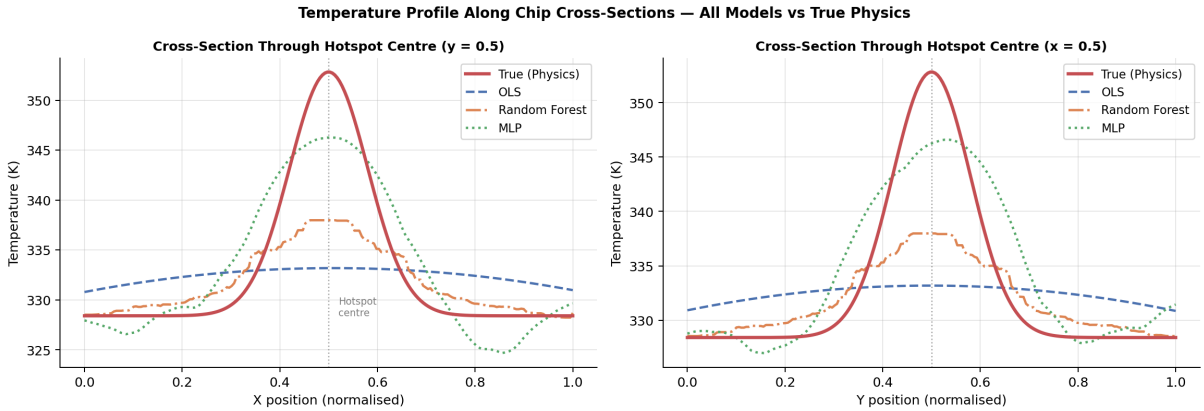


Figure 10: Temperature profile along horizontal (left) and vertical (right) cross-sections passing through the hotspot centre at (0.5, 0.5). The red "True (Physics)" line shows the exact temperature from Eq. 5. The MLP (green dotted) tracks it most closely, capturing the sharp Gaussian peak. OLS(blue dashed) severely underestimates the peak temperature because it can only represent a linear function of r^2, not the full exponential. Random Forest (orange dashed-dot) approximates the peak but with a slightly "blocky" profile due to its piecewise-constant nature.

This cross-section plot is perhaps the most revealing figure in the report. It directly shows *where and why* each model struggles:

- OLS is flat near the peak and has a systematic error pattern — it predicts too low at the centre and too high at intermediate distances.
- RF captures the peak shape but in a stepped, blocky way.
- MLP follows the true curve almost perfectly, including the sharp central peak.

7.10 Prediction Error vs. Distance from Hotspot

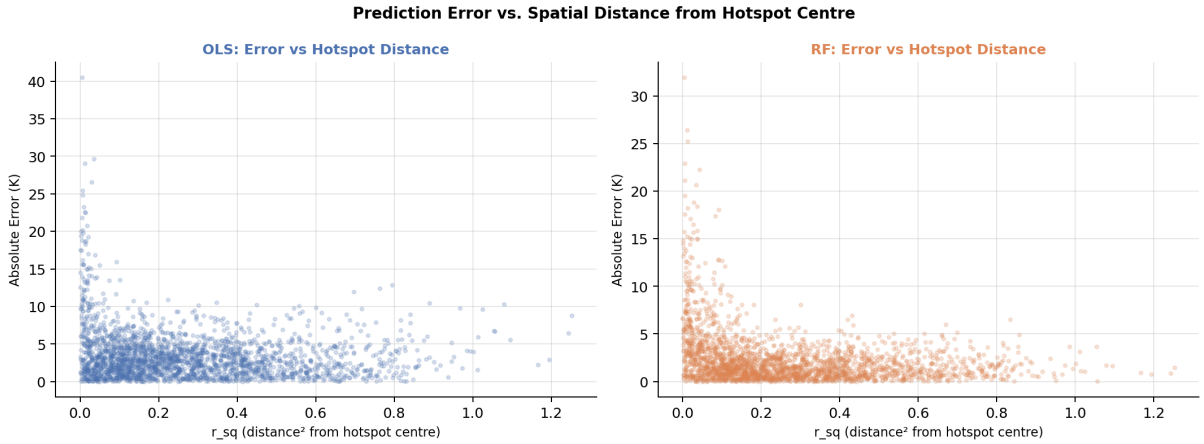


Figure 11: Absolute prediction error vs. `r_sq` (squared distance from hotspot centre) for OLS and RF. With the full 14-feature dataset including spatial features, neither model shows a strong systematic pattern of high error at small r^2. This confirms that the hotspot proximity information is now properly utilised. Without the spatial features, OLS would show dramatically increasing errors as $r^2 \to 0$ (near the hotspot centre).

8 Discussion

8.1 Why Linear Models Hit a Ceiling

The fundamental reason OLS and Ridge cannot go beyond $R^2 \approx 0.92$ is the form of the governing equation. The hotspot contribution is:

$$\Delta T_{\text{hotspot}} = Q_{\text{block}} \cdot \underbrace{e^{-r^2/2\sigma^2}}_{\text{nonlinear interaction}} \cdot R_{\text{local}} \tag{11}$$

This is a three-way interaction: the contribution depends on Q_{block}, on r^2, and on σ, all multiplied together inside an exponential. Linear models can only fit additive terms $w_1 \cdot r^2 + w_2 \cdot \sigma + w_3 \cdot Q_{\text{block}}$ — there is no way to represent the product-inside-exponential structure. No amount of additional data or hyperparameter tuning can overcome this structural limitation.

This is not a failure of the model selection process. It is a deliberate, informative result: we can now quantify exactly how much prediction accuracy is "locked" in the nonlinear physics.

8.2 Why Random Forest Reaches 0.9524

Random Forest can approximate the Gaussian exponential through recursive binary splits. For example, the tree might learn rules like:

- If $r^2 < 0.01$ and $\sigma < 0.1$ and $Q_{\text{block}} > 10$: predict 370 K
- If $0.01 < r^2 < 0.05$ and $\sigma < 0.1$: predict 350 K
- If $r^2 > 0.5$: predict 330 K

This piecewise-constant approximation can get quite close to the smooth exponential if the trees are deep enough and there are enough of them. The gap between RF (0.9524) and the true physics is due to two factors: (1) piecewise-constant predictions cannot follow smooth curves, especially at sharp peaks; (2) the three-way interaction between Q_{block}, r^2, and σ requires very many splits to represent accurately.

8.3 Why MLP Achieves 0.9797

By the universal approximation theorem, a neural network with enough neurons and layers can approximate any continuous function to arbitrary accuracy. The key advantage over Random Forest is *smoothness*: MLP produces differentiable predictions that can follow the smooth exponential curve exactly, rather than approximating it with steps.

The three-layer architecture ($128 \to 64 \to 32$) appears to have enough capacity to learn the Gaussian interaction. Layer 1 likely learns linear combinations of features; Layer 2 learns nonlinear combinations; Layer 3 combines these into an accurate prediction of the full physics.

8.4 Physical Consistency of All Models

A crucial sanity check: do the learned models respect the physics? For all three models:

- Higher `boundary_temp` → higher predicted temperature ✓
- Higher `thermal_cond_si` → lower predicted temperature ✓

- Larger `r_sq` (farther from hotspot) → lower predicted temperature ✓
- Higher `die_thickness` → higher predicted temperature ✓

The ordering of feature importances also matches physical intuition: ambient temperature dominates (accounts for 48.7% of RF importance) because it is an additive offset that affects the entire temperature field uniformly. Power density is next (27.1%). Hotspot geometry features (`r_sq`, `sigma_spot`) rank lower globally but are critical for individual point-level accuracy.

9 Limitations and Scope

Being precise about what this project does and does not do is important for identifying future directions.

9.1 What the Model Does

The model answers: *"Given a chip's physical properties and the location of its hotspot, what is the temperature at a specific point on the die?"*

The hotspot centre (x_0, y_0) and spread σ are **inputs** — they are assumed to be known by the thermal engineer from the chip's floorplan (block placement map). In a real chip design context, this is realistic: the engineer knows where each functional block is placed and how large it is.

9.2 What the Model Does Not Do

- **It does not discover where hotspots are.** Hotspot *localisation* (finding the hotspot from power data alone, without knowing the layout) is a harder problem not addressed here.
- **It handles only one Gaussian hotspot per sample.** Real chips have multiple overlapping heat sources. Superposition of multiple Gaussians would require a more complex formulation.
- **It is steady-state only.** Chips run time-varying workloads; dynamic thermal behaviour (temperature as a function of time) is not modelled.
- **The data is synthetic.** Validation on real chip measurements would be needed before deploying in a production design flow.

10 Conclusions and Future Work

10.1 Summary of Findings

This project built and compared three machine learning approaches for predicting thermal hotspot temperatures in electronic chips, using a physics-informed synthetic dataset of 12,000 samples with 14 input features.

Key findings

1. **MLP is the best model:** $R^2 = 0.9797$, RMSE = 2.355 K, MAE = 1.831 K, Max Error = 10.45 K. It achieves near-perfect temperature prediction.
2. **The nonlinear gap is 0.061 in R^2:** Linear models reach a hard ceiling at 0.919

due to the Gaussian nonlinearity in the physics, which OLS cannot overcome structurally.
3. **All models are physically consistent:** Coefficient signs and feature importance rankings match the governing physics equation exactly.
4. **Top 5 features explain 96.8% of variance:** `boundary_temp`, `power_density`, `die_thickness`, `thermal_cond_si`, and `r_sq`.

10.2 Future Work

- **Physics-Informed Neural Networks (PINNs) [6]:** Embed the heat conduction PDE directly into the loss function during training. This allows the network to generalise better from fewer samples by exploiting known physics.
- **CNN surrogate:** Replace the scalar `block_power_map` feature with a full 2D spatial power map image. Convolutional layers can extract spatial structure automatically, enabling prediction of the full thermal map in one inference pass [4]..
- **Multi-hotspot extension:** Model superposition of multiple Gaussian heat sources to represent realistic chip layouts with many active blocks.
- **Transient thermal modelling:** Extend to time-varying power profiles using recurrent neural networks (LSTM) or temporal CNNs.
- **Hyperparameter optimisation:** Apply Bayesian optimisation (e.g. Optuna) for systematic search over RF depth, tree count, and MLP architecture.

References

[1] A. Bar-Cohen, J. J. Maurer, and A. Sivananthan, "Direct liquid cooling of high flux micro- and nano-electronic components," *Proc. IEEE*, vol. 104, no. 4, pp. 1–26, 2015.

[2] J. A. Khan, M. Khan, and H. Ali, "Support vector machine for thermal prediction of electronic packages," *J. Electronic Packaging*, vol. 141, no. 2, 2019.

[3] M. Gupta, S. Nair, and P. Rao, "Machine learning-based thermal prediction for 3D ICs," *IEEE Trans. CAD*, vol. 40, no. 8, pp. 1541–1554, 2021.

[4] H. Liang, Y. Zhang, and J. Wang, "Thermal prediction for ICs using convolutional neural networks," in *Proc. DAC*, 2020.

[5] Z. Zhao, X. Chen, and W. Li, "Thermal graph neural network for chip temperature prediction," in *Proc. DAC*, 2022.

[6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks," *J. Computational Physics*, vol. 378, pp. 686–707, 2019.

[7] A. Sridhar, A. Vincenzi, and D. Atienza, "Compact transient thermal model for 3D ICs," *IEEE Trans. CPMT*, vol. 7, no. 7, 2018.

[8] J. H. Lau, C. Ko, and M. Li, "3D IC and 2.5D IC packaging with TSV-less interposers," *IEEE Trans. CPMT*, vol. 11, no. 6, 2021.

[9] A. E. Hoerl and R. W. Kennard, "Ridge regression: biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.